

PROJECT REPORT

Coursera AI/ML Course Advisor RAG Chatbot

Building a RAG-Powered Chatbot with Hybrid Q/A Retrieval

Course: AGAI-03 — Building Agentic AI applications for Beginners

Project Type: Solo Project

Target Website: coursera.org (AI/ML Courses)

Date: May 2026

1. CHOSEN WEBSITE & JUSTIFICATION

Website: *Coursera — AI & Machine Learning Courses*

URL: `https://www.coursera.org/courses?query=machine+learning`

Coursera was selected as the target website for the following reasons:

- **Rich structured content:** Each course page contains consistent metadata — title, description, instructor, rating, duration, skill tags, and level — making it ideal for a question-answering system.
- **High learner relevance:** Questions like "Which course should I start with?" or "How long does the Deep Learning Specialization take?" are naturally conversational and well-suited for a RAG-powered chatbot.
- **Diverse content depth:** Courses range from beginner (AI for Everyone) to advanced specializations (Deep Learning, NLP), providing variety across queries.
- **Publicly accessible metadata:** Despite heavy JavaScript rendering, Coursera embeds structured JSON-LD and Open Graph meta tags directly in HTML, allowing reliable server-side scraping without a headless browser.

25 course URLs were selected spanning machine learning fundamentals, deep learning, NLP, data science, and generative AI — covering a broad range of learner intents.

2. SCRAPING METHODOLOGY & CHALLENGES

Approach

Scraping was implemented using `requests` and `BeautifulSoup4` with a two-layer extraction strategy:

1. **JSON-LD structured data** (`<script type="application/ld+json">`) — The primary source for course title, description, rating, duration, instructor, and skills. Coursera embeds this for SEO purposes and it is reliably server-rendered.
2. **Open Graph meta tags** — Used as a fallback for title and description when JSON-LD is absent or incomplete.
3. **Raw visible text** — After stripping navigation, scripts, headers, and footers, up to 6,000 characters of body text were captured per page for richer document context.

Each scraped course was saved as an individual `.txt` file and aggregated into `all_courses.json` for downstream processing.

Challenges Faced

Challenge

Details

Mitigation

JavaScript-heavy rendering	Most course content loads client-side via React; the full page content is not in the initial HTML response	Relied on JSON-LD and meta tags which are always server-rendered
Anti-bot measures	Coursera rate-limits rapid requests; some pages returned 403 or incomplete responses	Added a 2-second delay between requests and used realistic browser User-Agent headers
Inconsistent page structure	Specialization pages differ structurally from individual course pages	Used a uniform extraction pipeline with graceful fallbacks at each field
Success rate	Only 18 of 25 targeted URLs yielded complete data	Failed pages were skipped and logged; 18 courses still provide sufficient coverage

3. Q/A GENERATION PROMPT STRATEGY

Synthetic Q/A pairs were generated using **Groq's LLaMA 3.3 70B Versatile** model. For each scraped course, the model was prompted with structured course metadata and instructed to produce realistic learner questions with accurate, grounded answers.

Prompt Design

You are an educational content expert. Based on the Coursera course information below, generate {num_pairs} diverse and realistic question-answer pairs that a student might ask.

Course Information:
{title, description, instructor, rating, level, duration, skills, raw_text}

Return ONLY a JSON array:
[{"question": "...", "answer": "..."}, ...]

- Guidelines:
- Questions must be natural and conversational
 - Answers must be 2-4 sentences, helpful, and specific to this course
 - Cover: content, difficulty, duration, prerequisites, skills, instructor, price
 - Include beginner-level questions
 - Include at least one question about who the course is recommended for

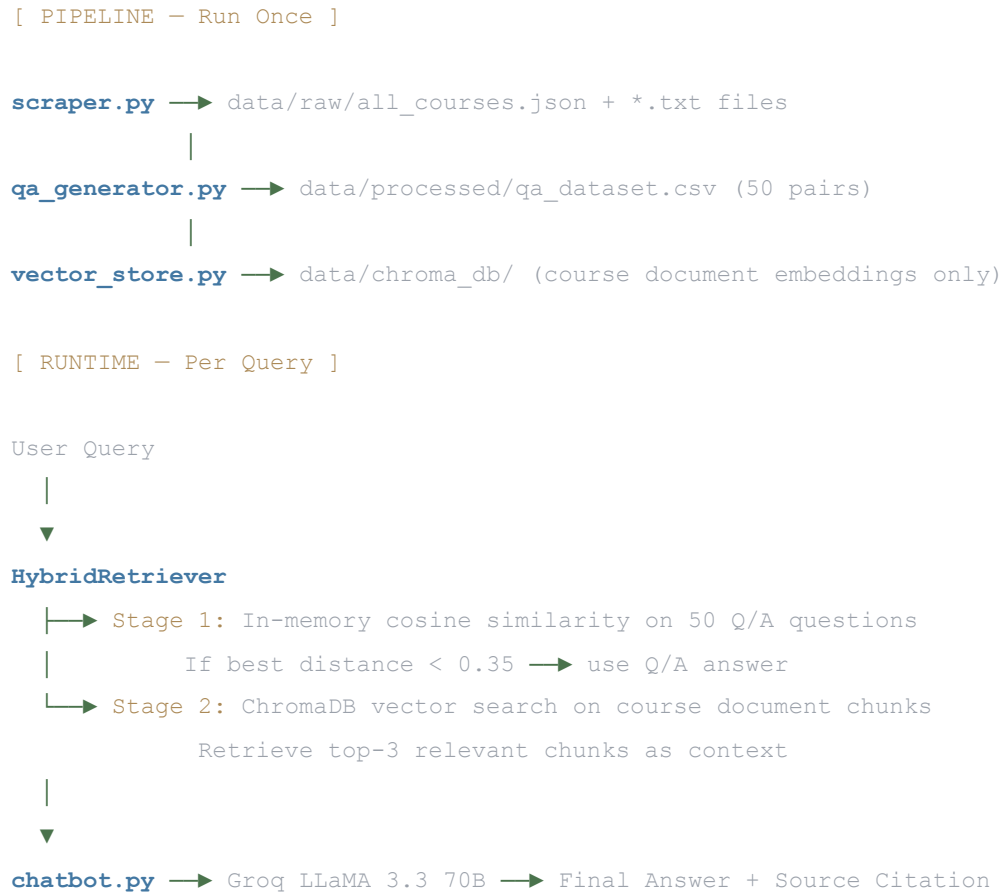
Design Decisions

- **Structured JSON output:** Enforcing a JSON array response eliminates parsing ambiguity and makes extraction reliable across all 25 courses.

- **Diversity constraint:** The prompt explicitly lists topic categories to prevent the model from generating repetitive variations of the same question.
- **Grounded answers:** By supplying raw page text (up to 1,200 characters) alongside structured metadata, answers are tied to actual course content rather than hallucinated.
- **Volume:** 2 pairs per course, capped at 50 total — sufficient for high-confidence Q/A matching without over-engineering a small dataset.

Temperature: 0.7 — balanced between creativity (varied question phrasing) and accuracy (factual answers tied to course data).

4. ARCHITECTURE



Component Breakdown

File	Role
scraper.py	Fetches and parses 25 Coursera course pages; outputs structured JSON + text files
qa_generator.py	Uses Groq LLaMA to generate 2 Q/A pairs per course (capped at 50)

vector_store.py	Chunks course documents, embeds with <code>all-MiniLM-L6-v2</code> , persists to ChromaDB
retriever.py	In-memory Q/A semantic search + ChromaDB document search; returns best context
chatbot.py	Formats context + conversation history into a prompt; calls Groq for final answer
app.py	Streamlit UI — header stats, sample questions, chat interface, retrieval details

5. HYBRID RETRIEVAL EXPLANATION

The retrieval system uses a two-stage approach designed to prioritise fast, precise answers from known Q/A pairs while maintaining broad coverage via document search.

Stage 1 — In-Memory Q/A Semantic Search

At startup, all 50 Q/A *questions* are embedded into vectors using `sentence-transformers (all-MiniLM-L6-v2)` and held in memory as a NumPy matrix. For each incoming query:

1. The query is embedded into the same vector space.
2. Cosine similarity is computed between the query vector and all 50 Q/A question vectors in a single matrix operation.
3. If the closest match has a cosine distance below **0.35**, the corresponding pre-written answer is used directly, supplemented with supporting document chunks.

Why in-memory instead of ChromaDB for Q/A? With only 50 pairs, the overhead of persisting to and querying a vector database is unnecessary. In-memory computation is faster, simpler, and avoids maintaining a second ChromaDB collection.

Stage 2 — ChromaDB Document Vector Search (Fallback)

If no Q/A pair meets the confidence threshold, the query is sent to ChromaDB, which holds chunked embeddings of all 18 scraped course documents. The top-3 most semantically similar chunks are retrieved and passed as context to the LLM.

Final Answer Generation

The retrieved context (Q/A answer or document chunks), the conversation history (last 3 exchanges), and the user's question are combined into a structured prompt and sent to **Groq LLaMA 3.3 70B Versatile** at temperature 0.5 for a focused, factual response.

Aspect	Q/A Match Path	Vector Search Path
--------	----------------	--------------------

Trigger	Cosine distance < 0.35	No confident Q/A match
Context source	Pre-written answer + 2 supporting chunks	Top-3 document chunks
Speed	Faster (in-memory)	Slightly slower (ChromaDB query)
Precision	High (exact-ish match)	Broader but less precise
UI indicator	 Q/A Match	 Vector Search

6. SCREENSHOTS OF WORKING CHATBOT

 Screenshot 1 — Main chat interface showing header, stats, and "Try asking" buttons

 **Coursera AI/ML Course Advisor**

Clear

AI-powered course advisor that helps you find the right AI/ML course on Coursera.

Pages Scraped

18

Q/A Pairs

32

[🔗 Coursera AI/ML Courses ↗](#)

Ask me anything about AI & Machine Learning courses on Coursera — from beginner to advanced!

>  Sample Questions

Ask about Coursera AI/ML courses...



 Screenshot 2 — A chat response showing "✅ Q/A Match" retrieval badge and source citation

I am a beginner willing to learn AI and LLMs can you suggest a course for me?



As a beginner, I recommend starting with the "AI For Everyone" course, which is designed for beginners with no prior experience required. This course covers the fundamentals of AI, including data ethics, machine learning, and deep learning, and can be completed at your own pace in around 7 hours. Once you have a solid foundation in AI, you can then consider moving on to the "Generative AI with Large Language Models" course to learn about the specifics of LLMs and how to deploy them in real-world applications. The "AI For Everyone" course is a great starting point, and it's available for free. This will provide a strong foundation for further learning, including the more specialized topic of LLMs.

▼ 🔍 Retrieval Details

Method: ✅ Q/A Match

Source: Q/A Dataset — Course: AI For Everyone | <https://www.coursera.org/learn/ai-for-everyone>

Ask about Coursera AI/ML courses...



[Screenshot 3 — A chat response using 'Vector Search' fallback](#)



What programming language does the data visualization course use?



The retrieved course information doesn't specifically mention a "data visualization course". However, the 'Tools for Data Science' course discusses various programming languages used in data science, including Python, R, SQL, Java, Scala, C++, JavaScript, and Julia. If you're looking for a course that covers data visualization, I recommend checking the course descriptions for 'Tools for Data Science' or 'What is Data Science?' to see if they cover data visualization and the programming languages used. Unfortunately, the provided information doesn't directly answer your question about the programming language used in a specific data visualization course.

▼ Retrieval Details

Method: Vector Search

Source: Vector Search on Course Documents

Ask about Coursera AI/ML courses...



7. LIMITATIONS & FUTURE IMPROVEMENTS

Current Limitations

- **JavaScript-limited scraping:** Coursera renders most content client-side. Only server-embedded JSON-LD and meta tags could be reliably extracted, missing dynamic sections like course syllabi and learner reviews.
- **Scrape success rate:** 7 of 25 URLs failed due to anti-bot rate limiting, resulting in 18 courses. The dataset is functional but not exhaustive.
- **Small Q/A dataset:** With 50 pairs across 18 courses (~2–3 per course), coverage of edge-case or niche questions is limited. Many queries fall through to vector search.
- **No real-time data:** The pipeline is run once. Course details (pricing, availability, new courses) can go stale without re-scraping.
- **Single retrieval stage shown to LLM:** Only the top match (Q/A or document) is used as primary context; a re-ranking step could improve answer quality for ambiguous queries.

Future Improvements

- **Selenium/Playwright integration** — Enable full JavaScript rendering to access syllabi, reviews, and learner testimonials for richer Q/A generation.
- **Expand to 50+ courses** — Broader coverage across specializations, professional certificates, and guided projects.
- **Cross-encoder re-ranking** — Add a second-pass model (e.g., `cross-encoder/ms-marco-MiniLM`) to re-score retrieved chunks before passing to the LLM, improving answer relevance.
- **Scheduled re-scraping** — Automate periodic pipeline runs to keep course data current.
- **User feedback loop** — Capture thumbs up/down per response to identify retrieval gaps and improve the Q/A dataset iteratively.

8. LEARNINGS

- **RAG architecture in practice:** Building this end-to-end made the RAG pipeline tangible — the interplay between retrieval quality and LLM output quality is direct. A good retrieval step matters more than prompt engineering the LLM.
- **Scraping real websites is messy:** Coursera's JS-heavy architecture meant that standard scraping approaches fail for most content. Learning to rely on structured data (JSON-LD) rather than raw DOM parsing was a key takeaway.
- **Right tool for the right scale:** Initially Q/A pairs were stored in ChromaDB. For a 50-item dataset, that's over-engineering — switching to in-memory cosine similarity made the system simpler and equally effective, reinforcing the principle of matching tool complexity to data scale.
- **LLM prompt design matters:** Small changes to the Q/A generation prompt (adding topic diversity constraints, enforcing JSON output) significantly improved the quality and variety of generated pairs.
- **Hybrid retrieval is powerful:** Using a fast, precise Q/A layer as the first pass and a broader document search as fallback showed how combining strategies outperforms either approach alone.